

Sistemas Operativos: Práctica 3

Shaofan Xu Javier Maria Santa

May 3, 2026

1 Introducción

En esta práctica el objetivo es ampliar la simulación de minería de blockchain introduciendo mecanismos avanzados de comunicación entre procesos (IPC) y memoria compartida. En concreto, se centraliza la salida de los mineros a través de un sistema de monitorización basado en colas de mensajes y un esquema productor-consumidor.

2 Requisitos cumplidos y no cumplidos

2.1 Requisitos cumplidos

Se han implementado correctamente los siguientes objetivos principales de la práctica, correspondientes a los apartados a y b:

- **Sistema de Monitorización (Apartado a):** Se ha desarrollado el ejecutable `monitor`, el cual exige estar arrancado antes que cualquier minero. Este consta de dos procesos que gestionan la salida unificada:
 - **Comprobador (Padre):** Recibe los bloques ganadores a través de una cola de mensajes POSIX (`/mq_monitor`). Antes de procesarlos, verifica su validez comprobando que `pow_hash(solucion) == target`. Si es válido, actúa como *Productor* e introduce el bloque en memoria compartida.
 - **Monitor (Hijo):** Actúa como *Consumidor*, leyendo los bloques de la memoria compartida e imprimiéndolos por pantalla respetando los *lags* de tiempo exigidos en la invocación.
- **Esquema Productor-Consumidor IPC:** La comunicación interna del Monitor se ha sincronizado empleando un segmento de memoria compartida (`shm_open` y `mmap`) que alberga un búfer circular. El acceso concurrente a dicho búfer está protegido mediante tres semáforos POSIX con nombre: `mutex`, `empty` y `full`.
- **Adaptación del Minero (Apartado b):** El proceso Minero envía un mensaje con su `pid`, `ronda`, `target` y solución por la cola al proclamarse ganador de una votación. Asimismo, cuando el tiempo del minero expira y detecta que es el último en salir, envía un bloque especial de terminación (`{-1, -1}`) por la cola.
- **Limpieza controlada de recursos:** Al recibir el bloque de finalización del último minero, el Comprobador intercepta el mensaje y se

encarga de realizar las llamadas `mq_unlink`, `shm_unlink` y `sem_unlink`, cerrando a su vez a su proceso hijo. Esto garantiza que no existan fugas de memoria en las estructuras del sistema operativo, como se ha comprobado en los test con Valgrind.

2.2 Requisitos no cumplidos

Todos los requisitos obligatorios especificados en el enunciado han sido implementados y funcionan correctamente.

3 Pruebas: Funcionamiento y Rendimiento

3.1 Entorno de ejecución

Las pruebas y mediciones se han llevado a cabo bajo las siguientes condiciones de hardware y software para garantizar la consistencia de los resultados:

- Sistema Operativo: WSL2 (Ubuntu) sobre Windows 11.
- Hardware: 16 GB de memoria RAM.
- Herramienta de medición: Comando `time` de la shell de Linux.

3.2 Pruebas de Funcionamiento

Se han ejecutado diversas pruebas para asegurar el correcto funcionamiento del programa.

3.2.1 Casos normales

Se ejecutó el programa con parámetros correspondientes (ej. `target` inicial 100, 10 rondas y 4 hilos):

```
./miner 100 10 4
```

El programa generó correctamente el archivo `.log` con la estructura especificada (Id, ronda, `target`, solución, etc), alternando la validación (`accepted/rejected`). Al finalizar, se imprime en la terminal exactamente los mensajes requeridos:

```
Solution accepted: 100 —> 2813411
Solution rejected: 2813411 —> 4952263
Solution rejected: 4952263 —> 1751421
Solution rejected: 1751421 —> 4619098
```

```
Solution accepted: 4619098 —> 9959801
Solution rejected: 9959801 —> 297242
Solution accepted: 297242 —> 7642589
Solution rejected: 7642589 —> 1566555
Solution rejected: 1566555 —> 7848135
Solution rejected: 7848135 —> 5915685
Logger exited with status 0
Miner exited with status 0
```

3.2.2 Casos de errores

Se probaron situaciones anómalas como:

- **Falta de argumentos:** Al ejecutar `./miner` sin parámetros, el programa detecta el error, muestra el uso correcto “*Usage ./miner < TARGET_INI > < ROUNDS > < N_THREADS >*” e imprime “*Miner exited unexpectedly*”.

3.3 Pruebas de Rendimiento

Para evaluar la eficiencia de la paralelización implementada, se han realizado pruebas de rendimiento variando el número de hilos de ejecución. Estas pruebas permiten analizar cómo el sistema escala al dividir el espacio de búsqueda entre múltiples unidades de procesamiento.

3.3.1 Pruebas realizadas

Se ejecutó el comando `./miner 0 10 N` (donde N es el número de hilos) manteniendo fijos el objetivo inicial (0) y el número de rondas (10).

Figure 1: Resultados de ejecución del algoritmo POW con diferentes hilos.

3.3.2 Resultados Obtenidos

Los datos obtenidos se resumen en la siguiente tabla:

Hilos (N)	Tiempo Real (s)	Tiempo User (s)	Tiempo Sys (s)	Speedup (S_n)
1	0.166s	0.075s	0.000s	1.00x (Base)
2	0.107s	0.046s	0.007s	1.55x
4	0.074s	0.010s	0.003s	2.24x
8	0.087s	0.094s	0.000s	1.90x

Table 1: Tiempos de ejecución y Speedup en función del número de hilos

3.3.3 Análisis y Razonamiento

Tras analizar los datos de la tabla, se pueden extraer las siguientes conclusiones técnicas:

- Paralelización efectiva (1 a 4 hilos): Se observa una reducción notable en el tiempo Real a medida que aumentamos los hilos hasta 4. Esto confirma que la estrategia de fragmentar el intervalo POW_LIMIT permite que los hilos realicen búsquedas de fuerza bruta de forma independiente y concurrente, optimizando el uso de los núcleos de la CPU.
- Saturación y Overhead (8 hilos): Al incrementar el número de hilos a 8, el tiempo Real aumenta ligeramente a 0.087s y el tiempo User se eleva significativamente a 0.094s. Este fenómeno ocurre porque, para una carga de trabajo de solo 10 rondas, el coste temporal de crear, sincronizar y destruir 8 hilos mediante `pthread_create` y `pthread_join` supera el beneficio obtenido por la computación paralela.
- Número óptimo de hilos: Para la arquitectura del sistema utilizado (WSL2) y la carga de trabajo específica de la práctica, el número óptimo se sitúa en 4 hilos. En este punto se alcanza el mejor equilibrio entre el aprovechamiento de la CPU y la sobrecarga mínima del sistema operativo en la gestión de hilos y cambios de contexto.
- Consumo de CPU: El hecho de que el tiempo User (tiempo total de CPU) sea superior al tiempo Real en la prueba de 8 hilos refleja que el sistema está invirtiendo más esfuerzo en la gestión de la concurrencia que en la resolución efectiva del problema matemático.

4 Conclusión y Respuesta a las cuestiones del enunciado

Pregunta d) ¿Qué conclusiones se pueden tomar respecto al número óptimo de hilos?

Una vez concluido todos los trabajos realizados, podemos concluir que el número óptimo de hilos coincide con el número de procesadores disponible para máquina.

Como se observa en la tabla, el máximo rendimiento se alcanza exactamente al utilizar 4 hilos. Al incrementar el número a 8 hilos, el rendimiento disminuye.

Ya que si creamos más hilos que el número del procesador, no es posible ejecutar los hilos de forma simultánea. El planificador del sistema operativo se va a obligar intercalando su ejecución.